



# MySQL变量、触发器和存储过程

“A good schema lasts longer than a good application.”

井明

数据科学与计算机学院

jingming@sdu.edu.cn

2025年春



山东女子学院  
Shandong Women's University

# 变量

什么是MySQL变量？

变量： 用于临时存储值，在 SQL 中进行计算、传递、控制流程等操作。

MySQL支持三种类型的变量：

| 变量类型   | 特点               | 示例                   |
|--------|------------------|----------------------|
| 用户定义变量 | 临时、当前会话可见        | SET @my_var = 10;    |
| 局部变量   | 存储过程或函数中定义，作用域局部 | DECLARE x INT;       |
| 系统变量   | 数据库系统的配置参数       | SELECT @@autocommit; |

## 用户定义变量（会话变量）

### ✓ 定义和使用方法：

```
SET @x = 100;  
SELECT @x;  -- 输出 100
```

### ✓ 特点：

- 以 @ 开头
- 当前会话有效，关闭连接即失效
- 无需预先声明，使用即定义

### ✓ 用途：

- 临时存值
- 传参给查询、函数
- 在触发器或存储过程外部传入值

🧠 局部变量（存储过程内部）

✓ 声明方法：

```
DECLARE my_var INT DEFAULT 10;
```

✓ 使用流程：

```
BEGIN  
  DECLARE sum INT;  
  SET sum = 10 + 20;  
  SELECT sum;  
END;
```

✓ 特点：

- 仅在 BEGIN...END 块中有效
- 不能用 @ 前缀
- 使用 DECLARE 语句定义

## ⚙️ 系统变量

### ✅ 查询系统变量

```
SHOW VARIABLES LIKE 'autocommit';  
SELECT @@autocommit;
```

### ✅ 修改系统变量

```
SET @@autocommit = 0;
```

### ✅ 特点:

- 控制MySQL服务器行为
- 有些变量是只读的
- 可用于优化和调试

## 变量应用实例

### 示例：使用变量求和

```
SET @a = 10, @b = 20;  
SELECT @a + @b AS total;
```

### 示例：将变量用于WHERE子句

```
SET @threshold = 500;  
SELECT * FROM orders WHERE amount > @threshold;
```

### 课堂练习题

1. 定义两个变量 **价格@price** 和 **数量@qty**，计算总价。
2. 在存储过程中声明一个变量 result，计算输入数字的平方。
3. 查询并设置系统变量 sql\_mode。

## 练习答案（参考）

1. 定义两个变量 @price 和 @qty，计算总价。

```
SET @price = 100.5, @qty = 3;  
SELECT @price * @qty AS total_price;
```



2. 在存储过程中声明一个变量 result，计算输入数字的平方。

```
DELIMITER //
CREATE PROCEDURE SquareNum(IN num INT)
BEGIN
    DECLARE result INT;
    SET result = num * num;
    SELECT result;
END;
//
DELIMITER ;
```

### 3. 查询并设置系统变量 sql\_mode。

```
SHOW VARIABLES LIKE 'sql_mode';  
SET @@sql_mode = '';
```

## 课堂小结

| 变量类型 | 标志  | 作用域   | 使用场景      |
|------|-----|-------|-----------|
| 用户变量 | @   | 当前会话  | 查询传参、中间计算 |
| 局部变量 | 无前缀 | 存储过程中 | 控制流程、临时值  |
| 系统变量 | @@  | 全局或会话 | 参数配置      |

### 课后思考题

1. 用户变量和局部变量有什么本质区别？
2. 如何在触发器中使用变量传递外部信息？
3. 系统变量修改对所有连接是否都生效？为什么？

## 课后思考题参考答案

? 1. 用户变量和局部变量有什么本质区别?

| 项目   | 用户变量 ( @ )                      | 局部变量 ( DECLARE )             |
|------|---------------------------------|------------------------------|
| 声明方式 | 直接赋值即可                          | 必须使用 <code>DECLARE</code> 声明 |
| 使用范围 | 当前会话全局                          | 仅限存储过程/函数内部                  |
| 生命周期 | 会话级, 连接关闭失效                     | 块级, 代码块执行完即失效                |
| 前缀   | 必须以 <code>@</code> 开头           | 不需要前缀                        |
| 初始化  | 可不初始化 ( 默认为 <code>NULL</code> ) | 必须显式声明或初始化                   |

📌 小结: 用户变量适合跨语句临时传值; 局部变量用于结构化过程控制。

## ? 2. 如何在触发器中使用变量传递外部信息?

方法一：使用用户变量 @变量名，由外部 SQL 语句或应用设置

```
-- 设置操作人和IP  
SET @current_user = 'teacher1';  
SET @client_ip = '192.168.1.2';
```

触发器中使用该变量记录信息

```
INSERT INTO audit_log (user_name, ip_addr) VALUES (@current_user, @client_ip);
```

📌 注意：

- 用户变量在触发器中可用；
- 局部变量不适用于触发器（除非用于存储过程中）；
- 应用程序应先设置用户变量，再执行 DML 操作以激活触发器。

### ? 3. 系统变量修改对所有连接是否都生效？为什么？

取决于作用域，系统变量有两种作用域：

| 类型  | 修改方法              | 影响范围           |
|-----|-------------------|----------------|
| 会话级 | SET @@SESSION.xxx | 仅对当前连接生效       |
| 全局级 | SET @@GLOBAL.xxx  | 影响所有新建连接（当前不变） |

✓ 示例：

```
SET @@SESSION.sql_mode = 'STRICT_TRANS_TABLES'; -- 当前会话生效
SET @@GLOBAL.max_connections = 200;              -- 所有新连接生效
```

📌 小结：会话变量即时生效但局限于当前连接；全局变量修改后需新连接才生效（或重启服务）。

# 触发器 ( Trigger )

## 1. 概念

**触发器 ( Trigger )** 是与表有关的数据库对象，当对该表进行INSERT、UPDATE或DELETE等操作时，会自动触发并执行触发器中定义的SQL语句。

## 2. 用途

- 实现数据变更的自动处理
- 数据校验（如自动记录日志或审核）
- 保证数据一致性和完整性（如自动更新相关表）



### 3. 原理

触发器是一种 事件驱动机制，在以下三类事件中执行：

- BEFORE INSERT / UPDATE / DELETE
- AFTER INSERT / UPDATE / DELETE

用户定义好触发时机和要执行的语句，MySQL会在指定事件发生时自动执行这些语句。

## 4. 语法

✚ MySQL 触发器的语法结构:

```
CREATE TRIGGER trigger_name  
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}  
ON table_name  
FOR EACH ROW  
BEGIN  
    -- SQL语句块  
END;
```

✓ 关键部分解释:

| 部分                       | 含义                      |
|--------------------------|-------------------------|
| CREATE TRIGGER           | 创建一个新的触发器               |
| trigger_name             | 触发器名称（全局唯一）             |
| BEFORE / AFTER           | 表示触发器在操作前或操作后执行         |
| INSERT / UPDATE / DELETE | 指定哪种类型的数据操作将触发          |
| ON table_name            | 指定在哪张表上定义触发器            |
| FOR EACH ROW             | 每条受影响的记录执行一次触发器         |
| BEGIN ... END            | 触发器内可执行的SQL语句块（可包含多条语句） |

📌 可使用的伪记录:

- NEW.column\_name: 表示插入或更新后的新值
- OLD.column\_name: 表示更新或删除前的旧值

## 🎯 示例 1：插入前验证价格非负

```
CREATE TRIGGER check_price
BEFORE INSERT ON products
FOR EACH ROW
BEGIN
    IF NEW.price < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = '价格不能为负数';
    END IF;
END;
```

### 🔒 注意事项：

- 一个表的同一操作类型（如 BEFORE INSERT）最多只能有一个触发器；
- BEFORE触发器常用于数据校验和修正；
- AFTER触发器常用于日志记录、同步数据等操作；
- 触发器中不能使用 CALL 调用另一个存储过程修改同一张表，容易引起递归或死锁。

## 第一步：创建 products 表结构

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  price DECIMAL(10, 2) NOT NULL  
);
```

## ⚙️ 第二步：创建触发器（验证价格非负）

```
DELIMITER //
```

```
CREATE TRIGGER check_price  
BEFORE INSERT ON products  
FOR EACH ROW  
BEGIN  
    IF NEW.price < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = '价格不能为负数';  
    END IF;  
END;  
//
```

```
DELIMITER ;
```

### 第三步：插入测试数据

#### 合法数据（成功插入）

```
INSERT INTO products (name, price)
VALUES ('耳机', 199.99);
```

#### 查看插入成功的数据

```
SELECT * FROM products;
```

✗ 非法数据（会触发异常）

```
INSERT INTO products (name, price)
VALUES ('显卡', -500.00);
```

★ 执行后将返回错误：

ERROR 1644 (45000): 价格不能为负数



## 示例2

场景：订单表操作时记录日志

```
CREATE TABLE order_log (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT,  
    operation VARCHAR(50),  
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TRIGGER trg_after_order_insert  
AFTER INSERT ON orders  
FOR EACH ROW  
INSERT INTO order_log(order_id, operation)  
VALUES (NEW.id, 'INSERT');
```

## 第一步：创建订单表 orders

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  amount DECIMAL(10, 2),  
  order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

 第二步：创建日志表 order\_log

```
CREATE TABLE order_log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT,  
    operation VARCHAR(50),  
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

### ⚙️ 第三步：创建触发器（记录插入日志）

```
DELIMITER //
```

```
CREATE TRIGGER trg_after_order_insert  
AFTER INSERT ON orders  
FOR EACH ROW  
BEGIN  
    INSERT INTO order_log(order_id, operation)  
    VALUES (NEW.id, 'INSERT');  
END;  
//
```

```
DELIMITER ;
```

#### 第四步：插入测试数据

插入订单

```
INSERT INTO orders (customer_name, amount)
VALUES ('张三', 300.00),
       ('李四', 500.00);
```

## 🔍 第五步：查看结果

查看订单表：

```
SELECT * FROM orders;
```

查看日志表（验证是否记录日志）：

```
SELECT * FROM order_log;
```

你将看到 order\_log 中自动记录了每条订单的插入操作。

### 🎯 示例3

场景：订单表操作时记录日志，包含操作人、IP 和修改前的数据

✅ 目标：当订单表被 **更新** 时，我们希望记录以下内容到日志表：

- 操作类型（UPDATE）
- 订单 ID
- 操作人（假设用 session 变量传递）
- 客户端 IP（模拟字段）
- 修改前的数据（客户名、金额）
- 操作时间

## 第一步：修改订单表 orders

我们假设不做变化，仍然使用如下结构：

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  amount DECIMAL(10, 2),  
  order_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```



## 第二步：创建审计日志表 order\_audit\_log

```
CREATE TABLE order_audit_log (  
  log_id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT,  
  operation VARCHAR(10),          -- 比如 INSERT/UPDATE/DELETE  
  operated_by VARCHAR(50),        -- 模拟用户名  
  client_ip VARCHAR(20),          -- 模拟IP地址  
  old_customer_name VARCHAR(100),  -- 修改前客户名  
  old_amount DECIMAL(10, 2),       -- 修改前金额  
  log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

### ⚙️ 第三步：创建触发器（更新时记录日志）

```
DELIMITER //
```

```
CREATE TRIGGER trg_after_order_update
AFTER UPDATE ON orders
FOR EACH ROW
BEGIN
    INSERT INTO order_audit_log (
        order_id,
        operation,
        operated_by,
        client_ip,
        old_customer_name,
        old_amount
    )
    VALUES (
        OLD.id,
        'UPDATE',
        @current_user,      -- 假设由应用设置
        @client_ip,        -- 假设由应用设置
        OLD.customer_name,
        OLD.amount
    );
END;
//
```

```
DELIMITER ;
```

#### 第四步：模拟设置 session 变量并执行测试

-- 模拟当前登录用户和IP（在应用中由程序设定）

```
SET @current_user = 'admin_user';  
SET @client_ip = '192.168.1.101';
```

-- 插入初始数据

```
INSERT INTO orders (customer_name, amount) VALUES ('王五', 1000);
```

-- 执行更新操作（会触发审计日志记录）

```
UPDATE orders  
SET amount = 1200  
WHERE customer_name = '王五';
```

## 🔍 第五步：查看日志表

-- 查看审计日志表

```
SELECT * FROM order_audit_log;
```

你会看到一条 UPDATE 类型的记录，包含旧数据和操作信息。

## 总结扩展特性

| 记录内容   | 来源        | 技术方式                |
|--------|-----------|---------------------|
| 操作类型   | 固定字符串     | 'UPDATE' 或 'DELETE' |
| 操作用户   | Session变量 | @current_user       |
| 客户端 IP | Session变量 | @client_ip          |
| 修改前数据  | OLD 关键字   | OLD.column_name     |

## 课堂练习题

练习 1: 创建一个触发器, 当员工表 employees 有员工被删除时, 将其姓名和删除时间写入一个 deleted\_employees 表。

```
CREATE TABLE employees (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(50),  
  salary DECIMAL(10, 2),  
  department VARCHAR(50)  
);  
  
INSERT INTO employees (name, salary, department)  
VALUES  
  ('Alice', 8000.00, 'IT'),  
  ('Bob', 6000.00, 'Sales'),  
  ('Charlie', 7000.00, 'HR');
```

## 参考答案

```
CREATE TABLE deleted_employees (  
    name VARCHAR(100),  
    deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TRIGGER trg_before_employee_delete  
BEFORE DELETE ON employees  
FOR EACH ROW  
INSERT INTO deleted_employees(name)  
VALUES (OLD.name);
```

## 存储过程 ( Stored Procedure )



## 1. 概念

存储过程是预先编译好的SQL语句集合，存储在数据库中，可以通过调用过程名来重复执行。

## 2. 用途

- 简化复杂的逻辑操作
- 提高数据库操作的效率
- 实现业务逻辑封装
- 降低前端与数据库交互复杂度

## 3. 原理

存储过程像程序中的函数，包含输入参数、输出参数、局部变量、条件控制、循环等结构，支持完整的流程控制。调用时只需一次执行即可。

## 4. 语法

基本语法结构:

```
DELIMITER //
```

```
CREATE PROCEDURE procedure_name (  
    [IN|OUT|INOUT] param1 datatype,  
    [IN|OUT|INOUT] param2 datatype  
)  
BEGIN  
    -- SQL语句块  
END;  
//
```

```
DELIMITER ;
```

说明:

- procedure\_name: 存储过程名称
- 参数模式:
  - IN: 传入参数 (只读)
  - OUT: 输出参数 (返回值)
  - INOUT: 可读可写参数
- DELIMITER 是为了定义新的语句结束符, 避免; 提前终止 SQL

常用语法元素

| 关键词               | 功能说明   | 示例                             |
|-------------------|--------|--------------------------------|
| DECLARE           | 声明局部变量 | DECLARE x INT;                 |
| SET               | 设置变量值  | SET x = 10;                    |
| SELECT INTO       | 查询并赋值  | SELECT salary INTO x FROM emp; |
| IF ... THEN       | 条件语句   | IF x > 0 THEN ... END IF;      |
| WHILE/LOOP/REPEAT | 循环控制语句 | WHILE i < 10 DO ... END WHILE; |
| LEAVE             | 中断循环   | LEAVE loop_name;               |

## 6. 一个简单的存储过程示例

 任务：计算两个数的和并返回

```
DELIMITER //
```

```
CREATE PROCEDURE add_numbers(  
    IN num1 INT,  
    IN num2 INT,  
    OUT result INT  
)  
BEGIN  
    SET result = num1 + num2;  
END;  
//
```

```
DELIMITER ;
```

## 7、调用存储过程

```
-- 定义输出变量  
CALL add_numbers(5, 7, @sum);  
  
-- 查看结果  
SELECT @sum;
```

## 8. 示例

场景：根据部门ID查询该部门所有员工信息

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT)  
BEGIN  
    SELECT * FROM employees WHERE department_id = dept_id;  
END //
```

```
DELIMITER ;
```

```
-- 调用存储过程  
CALL GetEmployeesByDept(3);
```

## 9. 课堂练习题

练习 2：编写一个名为 AddBonus 的存储过程，向员工表中某员工的工资添加奖金。要求输入员工ID和奖金数。

## 10. 参考答案

```
DELIMITER //
```

```
CREATE PROCEDURE AddBonus(IN emp_id INT, IN bonus DECIMAL(10,2))  
BEGIN  
    UPDATE employees SET salary = salary + bonus  
    WHERE id = emp_id;  
END //
```

```
DELIMITER ;
```

```
-- 示例调用  
CALL AddBonus(1001, 1000.00);
```



🎓 小结：触发器 vs 存储过程

| 项目   | 触发器（Trigger）  | 存储过程（Stored Procedure） |
|------|---------------|------------------------|
| 触发方式 | 自动执行（由数据事件触发） | 手动调用                   |
| 使用场景 | 日志记录、自动验证     | 封装逻辑、批量处理              |
| 可控性  | 被动触发，无法直接调用   | 主动调用，支持复杂逻辑结构          |
| 参数支持 | 不支持参数         | 支持输入、输出、INOUT 参数       |